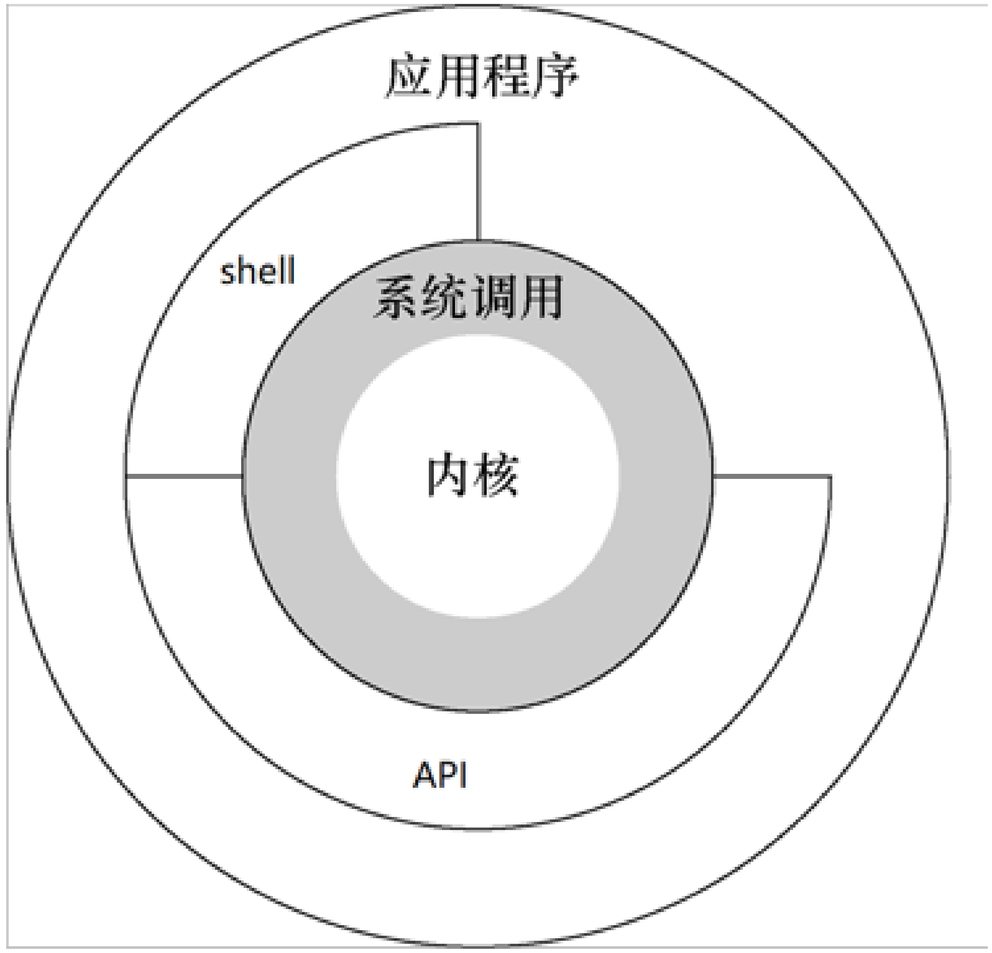
深入理解系统调用

- 系统调用概述
- 触发系统调用的方法
- 深入理解ARM64 Linux系统调用

系统调用

- 操作系统中一个非常重要的概念——系统调用。大多数程序员在写程序时都很难离开系统调用，与系统调用打交道的方式是通过库函数的方式，库函数用来把系统调用给封装起来，要理解系统调用的概念还需要一些储备知识



用户态与内核态

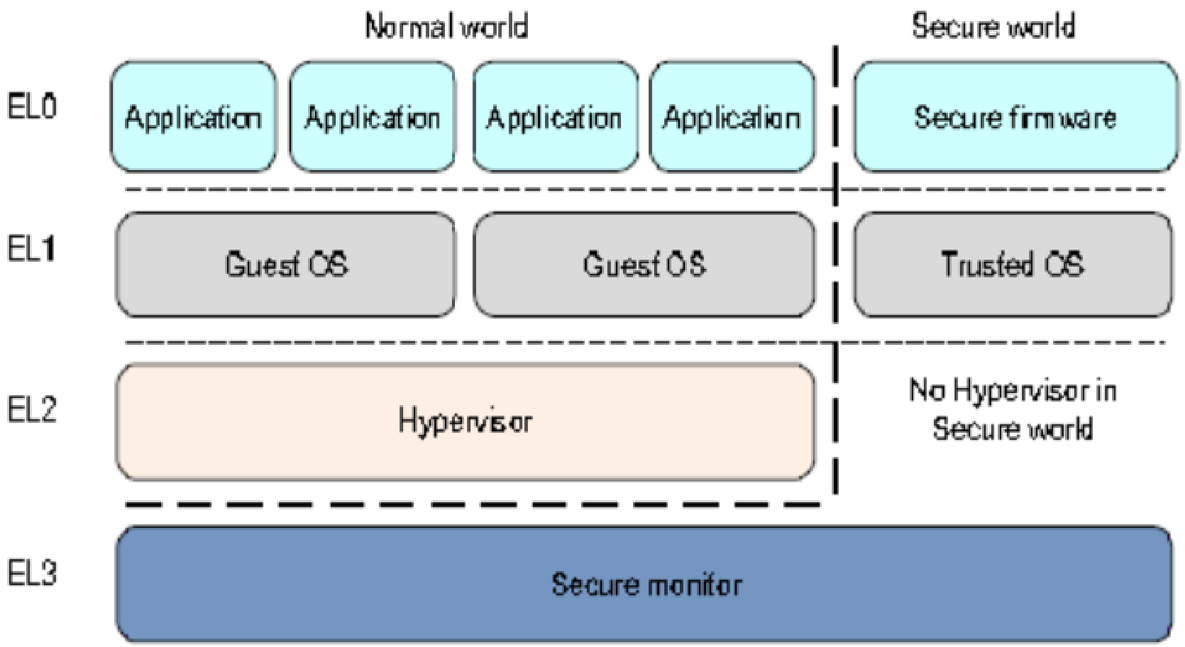
- 计算机的硬件资源是有限的，为了减少有限资源的访问和使用冲突，CPU 和操作系统必须提供一些机制对用户程序进行权限划分。现代的 CPU 一般都有几种不同的指令执行权限级别，在高的执行权限级别下，代码可以执行特权指令，比如访问任意内存，这时 CPU 的执行级别对应的就是内核态，所有的指令包括特权指令都可以执行。相应的，在用户态，代码（低权限级别指令）能够掌控的范围会受到限制，比如只能访问特定范围的内存。

- 为什么会出现这种情况呢？其实很容易理解，如果没有权限级别的划分，系统中不同程序员编写的应用程序都可以使用特权指令，由于不同的应用程序质量参差不齐，质量差的程序就很容易导致整个系统崩溃的。因为不是每个程序员写的代码都那么健壮，另外有些应用程序会非法访问其他进程甚至内核的资源，就会产生系统安全上的问题。

- 系统调用是操作系统发展的过程中为了保证系统稳定性和安全性的一种重要机制。系统调用的出现让普通程序员写的用户态的代码很难导致整个系统的崩溃，而操作系统内核的代码是由更专业的程序员写的，有规范的测试，相对就会更稳定、更健壮。

ARM64 CPU异常级别

- ARM64架构的CPU中指令执行权限级别被称为异常的级别（Exception Level，EL），也是分为4级，普通的用户程序处于EL0级，级别最低，Supervisor处于EL1级，Hypervisor处于EL2级，Secure monitor处于EL3级。Linux 操作系统在ARM64环境中只用了其中的EL0级作为用户态，EL1级作为内核态。



32 位 Linux 进程地址空间

- 在Linux系统中用户态和内核态很显著的区分方法就是指令指针寄存器的指向范围，在内核态下，指令指针寄存器的值可以是任意的地址，而用户态下指令指针寄存器的值只能访问受限的地址范围。

- 在 32 位Linux 系统上每个进程有 4GB 的进程地址空间，如右图所示。内核态下的这 4GB 的地址空间全都可以访问；但是在用户态时，只能访问 0x00000000 ~ 0xbfffffff 的地址范围，0xc0000000 以上的部分只能在内核态下访问。

64 位 Linux 进程地址空间

- 在Linux系统中用户态和内核态很显著的区分方法就是指令指针寄存器的指向范围，在内核态下，指令指针寄存器的值可以是任意的地址，而用户态下指令指针寄存器的值只能访问受限的地址范围。

- 在64位Linux 系统上，一般使用48位的地址总线，每个进程有256TB 的进程地址空间，如图所示。内核态下的这256TB的地址范围全都可以访问；但是在用户态时，只能访问0x000000000000 ~ 0x7ffffffff000 的地址范围，0x800000000000以上的部分只能在内核态下访问。

逻辑地址、线性地址和物理地址

- 这里所说的进程地址空间是进程的线性地址而不是物理地址。在操作系统原理中将地址分为逻辑地址、线性地址和物理地址。
- 逻辑地址一般时用段地址：段内偏移量来表示，起源 16 位X86 CPU 具有 20 位的地址总线可以访问的地址空间也就从 64K 扩展到 1M；
- 线性地址就是进程的地址空间里从 0 开始线性递增的地址空间范围里的地址，是逻辑地址经过段地址转换之后的地址；
- 物理地址就是实际物理内存的地址，线性地址经过内存管理单元（MMU）的内存页转换之后就是物理地址。
- 逻辑地址和线性地址在 32 位和 64 位上目前都是虚拟地址，需要依次经过分段、分页映射最后才转换成物理地址。这个映射计算地址的过程一般由 CPU 内部的 MMU负责把虚拟地址转换为物理地址。

系统调用是一种特殊的中断

- 系统调用最初是一种特殊的中断，一般中断（Interrupt）分外部中断和内部中断，内部中断又称为异常（Exception），异常又分为故障（fault）和陷阱（trap）。系统调用就是利用陷阱（trap）这种软件中断方式主动从用户态进入内核态的。
- 值得注意的是，在ARM64架构中，各种中断的划分与X86架构有所不同，比如中断被称为异常（Exception），又进一步分为同步异常（Synchronous）和异步异常，
- 异步异常又分为普通优先级的外部中断请求（IRQ）、高优先级的外部中断请求（FIQ）和系统错误（System Error）。
- 在程序运行过程中发生的异常事件，属于同步异常，这种异常的处理相当于同步阻塞方式调用了异常处理程序；在计算机运行过程中，出现某些意外情况需要干预时，CPU能自动停止正在运行的程序并转入处理新情况的程序，处理完毕后又返回原被暂停的程序继续运行，这些意外情况属于异步异常。
- 显然ARM64中的Exception对应Interrupt，Synchronous对应trap，System Error对应fault，Synchronous和System Error都属于内部中断，IRQ和FIQ属于外部中断。

系统调用的触发指令

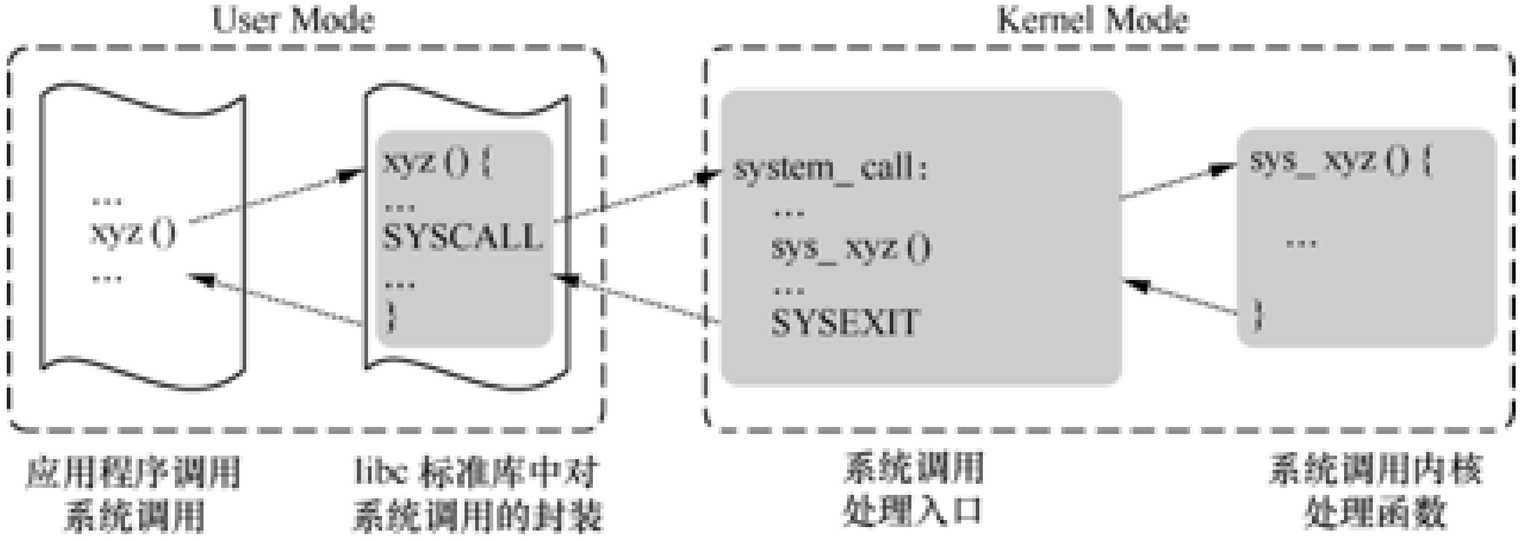
- 一般来说，从用户态进入内核态是由中断触发的，可能是外部中断，在用户态进程执行时，硬件中断信号到来，进入内核态，就会执行这个中断对应的中断处理程序。也可能是用户态程序执行过程中，调用了一个系统调用，陷入了内核态，叫作陷阱（trap）或同步异常（Synchronous）。

- 任何类型的中断处理都会有中断上下文的切换问题，当用户态切换到内核态时，就要把用户态程序执行的上下文环境保存起来，然后执行中断处理程序。

- 在ARM64架构下svc指令触发系统调用。一般系统调用会在堆栈上保存一些寄存器的值，会保存中断发生时当前执行程序的栈顶地址、当时的状态字、当时的指令指针寄存器的值。同时会将当前进程内核态的栈顶地址、内核态的状态字放入 CPU 对应的寄存器，并且指令指针寄存器的值会指向中断处理程序的入口，对于系统调用来讲是指向系统调用处理的入口。

系统调用的基本工作原理

- 系统调用是通过特定的软件中断向内核发出服务请求，比如ARM64架构svc指令的执行就会触发一个系统调用。系统调用返回指令，ARM64架构是异常返回指令eret



系统调用的意义

- 系统调用的意义是操作系统为用户态进程与硬件设备进行交互提供了一组接口。系统调用具有以下功能和特性。

- 把用户从底层的硬件编程中解放出来。操作系统为我们管理硬件，用户态进程不用直接与硬件设备打交道。

- 极大地提高系统的安全性。如果用户态进程直接与硬件设备打交道，会产生安全隐患，可能引起系统崩溃。

- 使用户程序具有可移植性。用户程序与具体的硬件已经解耦合并用接口代替了，不会有紧密的关系，便于在不同系统间移植。

Linux的系统调用号

- Linux内核中大约定义了四五百个系统调用，这时内核如何知道用户态进程希望调用的是哪个系统调用呢？内核通过给每个系统调用一个编号来区分，即系统调用号，将API函数xyz()和系统调用内核函数sys_xyz()关联起来了。内核实现了很多不同的系统调用，用户态进程必须指明需要执行哪个系统调用，这需要使用寄存器传递系统调用号。除了系统调用号外，系统调用也可能需要传递参数，

- ARM64是使用X8寄存器传递系统调用号

系统调用参数传递方式

- ARM64系统调用的参数传递是采用X0-X5这6个寄存器，系统调用号放在X8寄存器里传递。Linux系统调用最多有6个参数，ARM64函数调用参数可以使用X0-X7这8个寄存器。

ARM64 Linux系统调用概述

- 在ARM64架构下Linux系统调用由同步异常svc指令触发。当用户态（EL0级）程序调用库函数xyz()从而触发系统调用的时候，先把系统调用的参数依次放入X0-X5这6个寄存器（Linux系统调用最多有6个参数，ARM64函数调用参数可以使用X0-X7这8个寄存器），然后把系统调用号放在X8寄存器里，最后执行svc指令，CPU即进入内核态（EL1级）。顺便提一下svc指令一般会带一个立即数参数，一般是0x0，但并没有被Linux内核使用，而是把系统调用号放到了X8寄存器里。
- 在ARM64架构CPU中，Linux系统调用（同步异常）和其他异常的处理过程大致相同：
  - 1.异常发生时，CPU首先把异常的原因（比如执行svc指令触发系统调用）放在ESR_EL1寄存器里
  - 2.把当前的处理器状态（PSTATE）放入SPSR_EL1寄存器里；
  - 3.把当前程序指针寄存器（PC）放入ELR_EL1寄存器里，然后CPU通过异常向量表（vectors）基地址和异常的类型计算出异常处理程序的入口地址，即VBAR_EL1寄存器加上偏移量取得异常处理的入口地址，接着开始执行异常处理入口的第一行代码。
  - 4.这一过程是CPU自动完成的，不需要程序干预。

ARM64 Linux系统调用概述

- 进入异常处理入口之后，以svc指令对应的el0_sync为例，el0_sync处的内核汇编代码首先做的就是保存异常发生时程序的执行现场（保存现场），然后根据异常发生的原因（ESR_EL1寄存器）跳转到el0_svc，el0_svc处会根据系统调用号找到对应的系统调用内核处理函数，接着执行系统调用内核处理函数sys_syz()。

- 系统调用内核处理函数执行完成后，系统调用返回前，需要恢复异常发生时程序的执行现场（恢复现场），其中就包括主动设置ELR_EL1和SPSR_EL1的值，原因是异常会发生嵌套，一旦发生异常嵌套ELR_EL1和SPSR_EL1的值就会随之发生改变，所以当系统调用返回时，需要恢复之前保存的ELR_EL1和SPSR_EL1的值。最后内核调用异常返回指令eret，CPU自动把ELR_EL1写回PC，把SPSR_EL1写回PSTATE，并返回到用户态程序里，可以继续运行了。

触发系统调用的方法

- 以time系统调用为例，分别使用C库函数和int $0x80/syscall和svc指令汇编代码触发系统调用。

- C库函数 time_t time(time_t *seconds) 返回自1970-01-01 00:00:00（UTC国际时区）起经过的时间，以秒为单位。如果 seconds 不为空，则返回值也存储在变量 seconds 中。

- C库函数 struct tm *localtime(const time_t *timer) 使用 timer 的值来填充 stuct tm 结构体。timer 的值被分解到 stuct tm 结构体中，并用本地时区表示。

- 使用C库函数触发time系统调用的范例代码
- gcc -o time time.c -static
- objdump -S time > time64.S
- # 64位机器上编出32位代码需加 -m32
- sudo apt-get install gcc-multilib
- gcc -o time time.c -static -m32
- objdump -S time > time32.S

```
#include <stdio.h>
#include <time.h>
int main()
{
    time_t tt;
    struct tm *t;
    tt = time(NULL);
    t = localtime(&tt);
    printf("time: %d/%d/%d  %d:%d:%d\n",
            t->tm_year + 1900,
            t->tm_mon,
            t->tm_mday,
            t->tm_hour,
            t->tm_min,
            t->tm_sec);
    return 0;
}
```

ARM64汇编代码触发系统调用

- 因为在基于华为鲲鹏处理器的openEuler操作系统云主机环境下C库函数time内部使用的是gettimeofday系统调用，我们这里分别使用了gettimeofday库函数和内联ARM64汇编代码的方式触发系统调用了。
- 在标准库sys/time.h文件中定义了gettimeofday的函数原型如下：
- int gettimeofday(struct timeval*tv, struct timezone *tz);
- gettimeofday函数会把时间包装为timeval和timezone结构体返回，timeval中包括秒和微妙值，timezone中包括时区等信息。gettimeofday系统调用比time系统调用提供的时间信息更多也更精确，timeval结构体中的tv_sec是与time系统调用的返回值是相同的，我们这里只需要使用tv_sec的值。

- 在ARM64架构下Linux系统调用由同步异常svc指令触发。当用户态（EL0级）程序调用库函数gettimeofday从而触发系统调用的时候，先把系统调用的参数依次放入X0-X5这6个寄存器（这里仅需要使用X0和X1两个寄存器），然后把系统调用号放在X8寄存器里，最后执行svc指令，CPU即进入内核态（EL1级）。顺便提一下svc指令一般会带一个立即数参数，一般是0x0，但并没有被Linux内核使用，而是把系统调用号放到了X8寄存器里传递。

```
int main()
{
    time_t tt;
    struct timeval tv;
    struct tm *t;
#if 0
    gettimeofday(&tv,NULL);
#else
    asm volatile(
        "add   x0, x29, 16\n\t"  //X0寄存器用于传递参数&tv
        "mov   x1, #0x0\n\t"     //X1寄存器用于传递参数NULL
        "mov   x8, #0xa9\n\t"    //使用X8传递系统调用号169
        "svc   #0x0\n\t"         //触发系统调用
    );
#endif
    tt = tv.tv_sec;
    t = localtime(&tt);
    printf("time: %d/%d/%d %d:%d:%d\n",
        t->tm_year + 1900,
        t->tm_mon,
        t->tm_mday,
        t->tm_hour,
```

深入理解系统调用

- 系统调用的初始化，也就是将系统调用处理入口地址告诉CPU到哪里找
  - int $0x80是放到和其他中断一样按中断向量依次存放
  - sysenter和syscall都借助CPU内部的MSR寄存器来存放，所以查找系统调用处理入口地址会更快，因此也称为快速系统调用

深入理解系统调用

- 系统调用的执行，也就是用户程序触发系统调用之后，CPU及内核执行系统调用的过程
  - int $0x80是CPU压栈一些关键寄存器，接着内核负责保存现场，系统调用内核函数处理完后恢复现场，最后通过iret出栈哪些CPU压栈的关键寄存器。
  - sysenter和syscall都借助CPU内部的MSR寄存器来查找系统调用处理入口，可以快速切换CPU的指令指针（eip/rip）到系统调用处理入口，但本质上还是中断处理的思路，压栈关键寄存器、保存现场、恢复现场，最后系统调用返回。

以系统调用为例看中断上下文的切换

- 系统调用实质上是一种特殊的中断，int $0x80指令触发系统调用会在内核堆栈上保存一些寄存器的值，会保存系统调用发生时当前执行程序的栈顶地址（SS:ESP）、当时的状态字（EFlags）、当时的 CS:EIP的值。同时会将当前进程内核堆栈的栈顶地址、内核的状态字等放入 CPU 对应的寄存器，并且 CS:EIP 寄存器的值会指向中断处理程序的入口，对于系统调用来讲是指向系统调用处理的入口。

- 更一般地来看，中断发生时CPU第一时间就是保存当前CPU执行的关键上下文（栈顶指针寄存器、标志寄存器、指令指针寄存器等），然后保存现场就是把其他寄存器的值也保存起来，当中断处理程序结束时恢复现场并中断返回，也就时负责把中断时保存的“现场”恢复到当前的 CPU 里面。

- 最后的 iret 与中断信号（包括 int 指令）发生时的 CPU 做的动作正好相反，之前是保存，这里就是恢复。

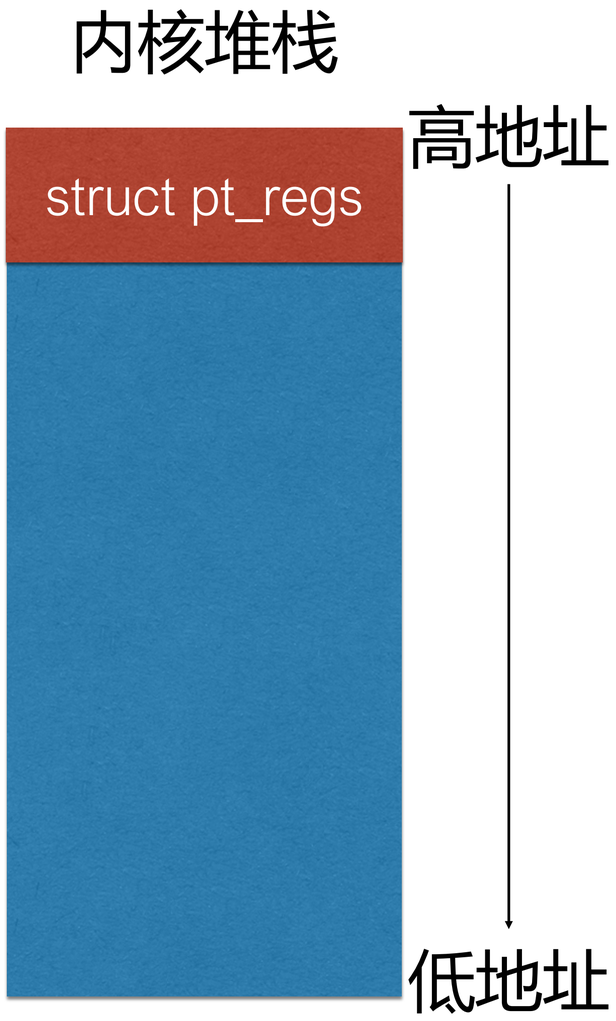
- sysenter/sysexit和syscall/sysret可以认为是一种更加特殊的中断机制。

系统调用处理入口

- do_int80_syscall_32和do_syscall_64的参数struct pt_regs *regs非常重要，它实际上是系统调用的内核堆栈栈底的一部分，也是中断上下文中保存现场和恢复现场所存储的关键数据。这里破坏了栈的逻辑结构，直接用struct pt_regs的结构体来操作它，因为需要有一些特殊处理来修改内核堆栈的栈底，这一点尤其值得关注。

系统调用的内核堆栈

```
struct pt_regs {
/*
 * C ABI says these regs are callee-preserved. They aren't saved on kernel entry
 * unless syscall needs a complete, fully filled "struct pt_regs".
 */
    unsigned long r15;
    …
    unsigned long r8;
    unsigned long ax;
    unsigned long cx;
    unsigned long dx;
    unsigned long si;
    unsigned long di;
/*
 * On syscall entry, this is syscall#. On CPU exception, this is error code.
 * On hw interrupt, it's IRQ number:
 */
    unsigned long orig_ax;
/* Return frame for iretq */
    unsigned long ip;
    unsigned long cs;
    unsigned long flags;
    unsigned long sp;
    unsigned long ss;
/* top of stack page */
}
```



系统调用过程总结

- int $0x80 或 syscall指令
- 系统调用处理入口entry_INT80_32 或 entry_SYSCALL_64
  - 保存现场
  - do_int80_syscall_32或do_syscall_64
    - 系统调用内核处理函数组成的ia32_sys_call_table和sys_call_table数组
    - 进程调度时机syscall_return_slowpath(regs)可以跟踪到schedule函数
  - 恢复现场
- 系统调用返回iret或sysret
- 继续执行int $0x80 或 syscall指令的下一条指令

深入理解ARM64 Linux系统调用

- ARM64 Linux异常向量表的初始化
- ARM64 Linux系统调用的执行
- ARM64 Linux系统调用内核处理函数

ARM64 Linux异常向量表的初始化

```
__primary_switched:
    adrp  x4, init_thread_union
    add   sp, x4, #THREAD_SIZE
    adr_l x5, init_task
    msr   sp_el0, x5            // Save thread_info

    adr_l x8, vectors           // load VBAR_EL1 with virtual
    msr   vbar_el1, x8          // vector table address
...
```

```
union thread_union {
#ifndef CONFIG_ARCH_TASK_STRUCT_ON_STACK
        struct task_struct task;
#endif
#ifndef CONFIG_THREAD_INFO_IN_TASK
        struct thread_info thread_info;
#endif
        unsigned long stack[THREAD_SIZE/sizeof(long)];
};
```

- 在start_kernel函数开始执行之前是用汇编代码初始化CPU，其中非常重要的就是将异常向量表的基地址（vectors）配置到VBAR_EL1寄存器中，从arch\arm64\kernel\head.S中可以找到代码，这段代码配置了不仅配置了异常向量表，还配置了0号进程的内核堆栈和进程描述符。

ARM64异常向量表

- ARM64架构中的中断向量表（Exception vectors）包含16个kernel_ventry，这16个kernel_ventry分为4组，每组包含4个kernel_ventry，参见/arch/arm64/kernel/entry.S，摘录代码如下，4组kernel_ventry以空行隔开。
- 对于Linux系统来说我们主要关注第2组和第3组。这4组中每组包含4个kernel_ventry分别对应4种类型的异常。

```
        .align      11
ENTRY(vectors)
        kernel_ventry   1, sync_invalid             // Synchronous EL1t
        kernel_ventry   1, irq_invalid              // IRQ EL1t
        kernel_ventry   1, fiq_invalid              // FIQ EL1t
        kernel_ventry   1, error_invalid        // Error EL1t

        kernel_ventry   1, sync                     // Synchronous EL1h
        kernel_ventry   1, irq                      // IRQ EL1h
        kernel_ventry   1, fiq_invalid              // FIQ EL1h
        kernel_ventry   1, error                // Error EL1h

        kernel_ventry   0, sync                     // Synchronous 64-bit EL0
        kernel_ventry   0, irq                      // IRQ 64-bit EL0
        kernel_ventry   0, fiq_invalid              // FIQ 64-bit EL0
        kernel_ventry   0, error                // Error 64-bit EL0

#ifdef CONFIG_COMPAT
        kernel_ventry   0, sync_compat, 32          // Synchronous 32-bit EL0
        kernel_ventry   0, irq_compat, 32           // IRQ 32-bit EL0
        kernel_ventry   0, fiq_invalid_compat, 32// FIQ 32-bit EL0
        kernel_ventry   0, error_compat, 32         // Error 32-bit EL0
#else
        kernel_ventry   0, sync_invalid, 32         // Synchronous 32-bit EL0
        kernel_ventry   0, irq_invalid, 32          // IRQ 32-bit EL0
        kernel_ventry   0, fiq_invalid, 32          // FIQ 32-bit EL0
        kernel_ventry   0, error_invalid, 32        // Error 32-bit EL0
#endif
END(vectors)
```

ARM64异常向量表

- 异常向量表分为4组，每组有4个向量入口地址，分别处理4种不同类型的异常。每个向量入口空间128字节，也就是说，在这个异常向量空间里可以放入32条指令（每条指令4字节）。举个例子，如果用户态程序执行了svc指令，这时CPU自动把VBAR_EL1寄存器的值（vectors），和第3组Synchronous的偏移量0x400相加，即vectors + 0x400，得出该异常向量空间的入口地址，然后跳转到那里，执行里面的第一条指令。

Address	Exception type	Description
+0x000	Synchronous	Current EL with SP0
+0x080	IRQ/vIRQ	
+0x100	FIQ/vFIQ	
+0x180	SError/vSError	
+0x200	Synchronous	Current EL with SPx
+0x280	IRQ/vIRQ	
+0x300	FIQ/vFIQ	
+0x380	SError/vSError	
+0x400	Synchronous	Lower EL using AArch64
+0x480	IRQ/vIRQ	
+0x500	FIQ/vFIQ	
+0x580	SError/vSError	
+0x600	Synchronous	Lower EL using AArch32
+0x680	IRQ/vIRQ	
+0x700	FIQ/vFIQ	
+0x780	SError/vSError	

ARM64异常向量表

- 这段代码，.align 7汇编伪代码，表示这一向量入口指令编译是以2的7次方对齐，也就是说每一个向量入口都有128字节存储空间，只要指令数量限制在128字节以内就可对应ARM64的向量偏移机制，正常的工作。
- 我们忽略条件编译部分的代码后，只剩下两句汇编指令。
- sub sp, sp, #S_FRAME_SIZE汇编代码是当前堆栈指针sp的内容减去S_FRAME_SIZE（值为sizeof(struct pt_regs)），由于Linux堆栈是向低地址方向发展的，所以此行代码的功能相当于在堆栈中拉出S_FRAME_SIZE大小的空间，用于保存现场（保存通用寄存器的值）。

```
    .macro kernel_ventry, el, label, regsize = 64
    .align 7
#ifdef CONFIG_UNMAP_KERNEL_AT_EL0
  ...
#endif

    sub    sp, sp, #S_FRAME_SIZE
#ifdef CONFIG_VMAP_STACK
  ...
#endif
    b   el\()\el\()_\label
    .endm
```

ARM64 Linux系统调用的执行

- 用户态程序执行svc指令，CPU会把当前程序指针寄存器PC放入ELR_EL1寄存器里，把PSTATE放入SPSR_EL1寄存器里，把异常产生的原因（这里是调用了svc指令触发系统调用）放在ESR_EL1寄存器里。这时CPU是知道异常类型和异常向量表的起始地址的，所以可以自动把VBAR_EL1寄存器的值（vectors），和第3组Synchronous的偏移量0x400相加，即vectors + 0x400，得出该异常向量空间的入口地址，然后跳转到那里执行异常向量空间里面的指令。每个异常向量空间仅有128个字节，最多可以存储32条指令（每条指令4字节），而且异常向量空间最后一条指令是b指令，对于系统调用来说会跳转到el0_sync，这样就从异常向量空间跳转同步异常处理程序的入口。

ARM64 Linux系统调用的执行

```
/*
 * EL0 mode handlers.
 */
el0_sync:
    kernel_entry 0
    mrs x25, esr_el1                // read the syndrome register
    lsr   x24, x25, #ESR_ELx_EC_SHIFT   // exception class
    cmp     x24, #ESR_ELx_EC_SVC64          // SVC in 64-bit state
    b.eq    el0_svc
    ...
ENDPROC(el0_sync)
```

- 值得注意的是el0_sync的第一句代码kernel_entry 0，这个kernel_entry和异常向量表的kernel_entry有所不同，因为它只有一个宏参数el，而异常向量表的kernel_entry有两个宏参数el和label，他们都包含默认宏参数regsize。

ARM64 Linux保存现场

在Linux系统中系统调用发生时，CPU会把当前程序指针寄存器PC放入ELR_EL1寄存器里，把PSTATE放入SPSR_EL1寄存器里，同时Linux系统从用户态切换到内核态（从EL0切换到EL1），这时SP指的是SP_EL1寄存器，用户态堆栈的栈顶地址依然保存在SP_EL0寄存器中。也就是说异常（这里是指系统调用）发生时CPU的关键状态sp、pc和pstate分别保存在SP_EL0、ELR_EL1和SPSR_EL1寄存器中。保存现场的主要工作如上代码所示，是保存x0-x30及sp、pc和pstate，这和struct pt_regs数据结构的起始部分正好一一对应。

```
.macro kernel_entry, el, regsize = 64
...
stp x0, x1, [sp, #16 * 0]
stp x2, x3, [sp, #16 * 1]
...
stp x26, x27, [sp, #16 * 13]
stp x28, x29, [sp, #16 * 14]
...
mrsx21, sp_el0
mrsx22, elr_el1
mrsx23, spsr_el1
stp     lr, x21, [sp, #S_LR]      // lr is x30
stp     x22, x23, [sp, #S_PC]
...
.endm
```

ARM64 Linux系统调用的执行

- el0_sync在完成保存现场的工作之后，会根据ESR_EL1寄存器确定同步异常产生的原因，同步异常产生的原因很多，在ARM64 Linux中最常见的原因是svc指令触发了系统调用，所以排在最前面的就是条件判断跳转到el0_svc，el0_svc中主要负责调用C代码的el0_svc_handler处理系统调用和ret_to_user系统调用返回。

- 假定系统调用处理完毕，ret_to_user系统调用返回相关的代码:

```
/*
 * SVC handler.
 */
    .align    6
el0_svc:
    gic_prio_kentry_setup tmp=x1
    mov     x0, sp
    bl    el0_svc_handler
    b     ret_to_user
ENDPROC(el0_svc)
```

```
asmlinkage void el0_svc_handler(struct pt_regs *regs)
{
    sve_user_discard();
    el0_svc_common(regs, regs->regs[8], __NR_syscalls, sys_call_table);
}
```

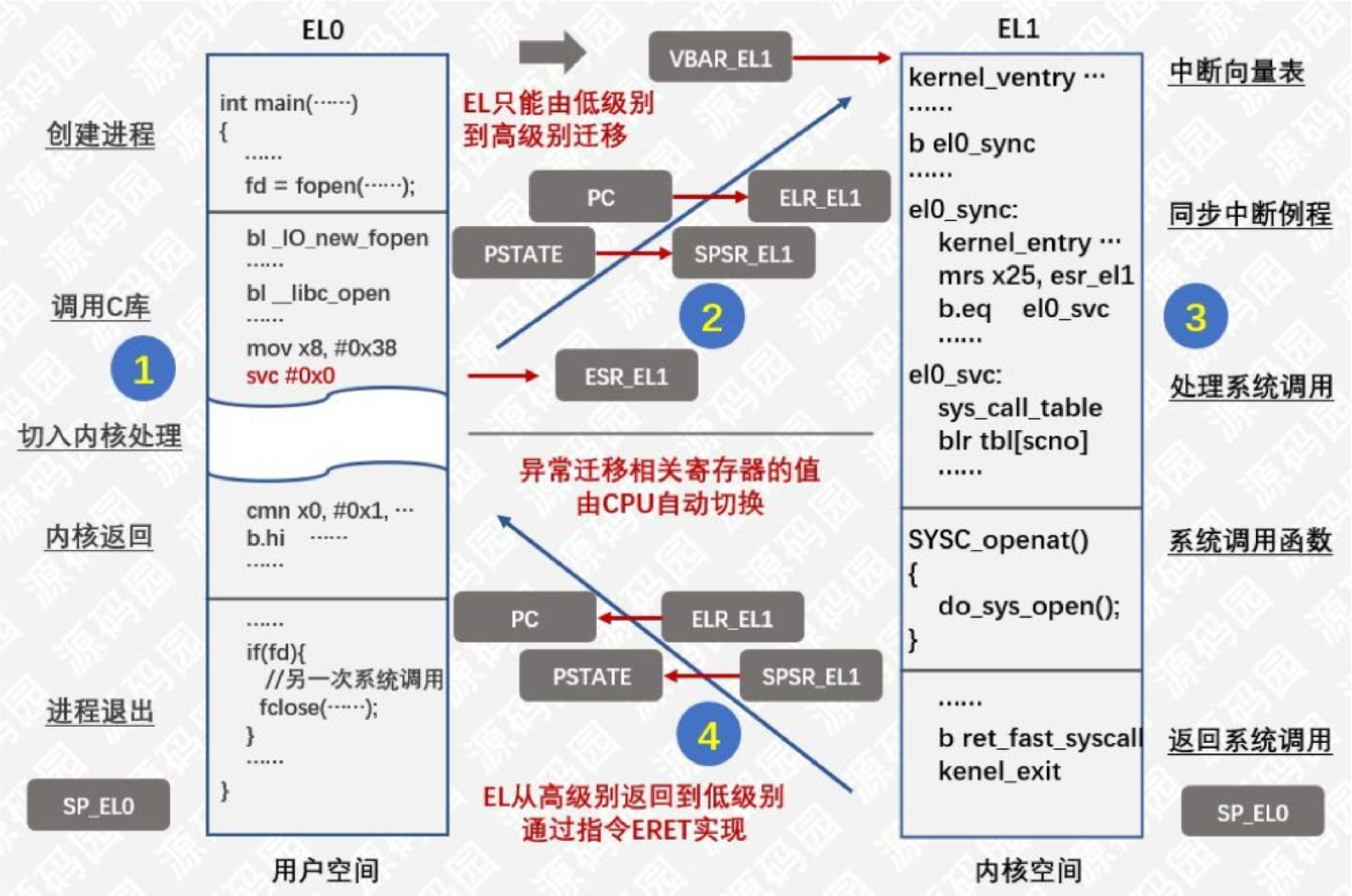
- 从如上代码中可以看到从系统调用返回前会处理一些工作（work_pending），比如处理信号、判断是否需要进程调度等，ret_to_user的最后是kernel_exit 0负责恢复现场，与保存现场kernel_entry 0相对应，kernel_exit 0的最后会执行eret指令系统调用返回。eret指令所做的工作与svc指令相对应，eret指令会将ELR_EL1寄存器里值恢复到程序指针寄存器PC中，把SPSR_EL1寄存器里的值恢复到PSTATE处理器状态中，同时会从内核态转换到用户态，在用户态堆栈栈顶指针sp代表的是sp_el0寄存器。

```
    bl   do_notify_resume
#ifdef CONFIG_TRACE_IRQFLAGS
    bl   trace_hardirqs_on       // enabled while in userspace
#endif
    ldr  x1, [tsk, #TSK_TI_FLAGS] // re-check for single-step
    b    finish_ret_to_user
/*
 * "slow" syscall return path.
 */
ret_to_user:
    disable_daif
    gic_prio_kentry_setup tmp=x3
    ldr  x1, [tsk, #TSK_TI_FLAGS]
    andx2, x1, #_TIF_WORK_MASK
    cbnz    x2, work_pending
finish_ret_to_user:
    enable_step_tsk x1, x2
#ifdef CONFIG_GCC_PLUGIN_STACKLEAK
    bl   stackleak_erase
#endif
    kernel_exit 0
ENDPROC(ret_to_user)
```

- kernel_exit 0负责恢复现场的代码和kernel_entry 0负责保存现场的代码相对应

```
.macro      kernel_exit, el
    ...
    msr      sp_el0, x23
    msr      elr_el1, x21              // set up the return
data
    msr      spsr_el1, x22
    ldp  x0, x1, [sp, #16 * 0]
    ldp  x2, x3, [sp, #16 * 1]
    ldp  x4, x5, [sp, #16 * 2]
    ldp  x6, x7, [sp, #16 * 3]
    ...
    ldp  x24, x25, [sp, #16 * 12]
    ldp      x26, x27, [sp, #16 * 13]
    ldp  x28, x29, [sp, #16 * 14]
    ldr  lr, [sp, #S_LR]
    ...
    eret
```

ARM64 Linux系统调用的执行



Linux内核分析
可执行程序工作原理

可执行程序工作原理

- ELF 目标文件格式
- 程序的编译过程
- 静态链接与动态链接
- execve系统调用的内核处理过程

ELF 目标文件格式

- 理解了进程的描述和创建之后，自然会想到我们编写的可执行程序是如何作为一个进程 工作的?这就涉及可执行文件的格式、编译、链接和装载等相关知识。

- 这里先提一个常见的名词“目标文件”，是指编译器生成的文件。“目标”指目标平台， 例如 x86 或 x86-64，它决定了编译器使用的机器指令集。目标文件一般也叫作 ABI(Application Binary Interface，应用程序二进制接口)，目标文件和目标平台是二进制兼容的。二进制兼容 即指该目标文件已经是适应某一种 CPU 体系结构上的二进制指令。例如一个编译出来的x86-64目标文件是无法链接成ARM上的可执行文件的。

- 最古老的目标文件格式是 a.out，后来发展成 COFF，现在常用的有 PE (Windows)和 ELF(Linux)。

- ELF(Executable and Linkable Format)即可执行的和可链接的格式， 是一个文件格式的标准。ELF 格式的文件用于存储 Linux 程序。ELF 是一 种对象文件的格式，用于定义不同类型的对象文件中都有什么内容、以什么样的格式放这些内容。ELF 首部会描绘整个文件的组织结构，它还包括很多节(sections，是在 ELF 文件里用以装载内容数据的最小容器)，这些节有些是系统定义好的，有些是用户在文件中通过.section 命令自定义的，链接器会将多个输入目标文件中相同的节合并。

ELF 概述

- ELF 文件的 3 种类型。以 ELF 格式为例，来看在可执行文件格式里的 3 种不同类型的目标文件。
- • 可重定位文件：这种文件一般是中间文件，还需要继续处理。由编译器和汇编器创建，一个源代码文件会生成一个可重定位文件。文件中保存着代码和适当的数据，用来和其他的目标文件一起来创建一个可执行文件或者动态链接库文件。在编译 Linux 内核时可能会注意到，每个内核源代码.c 文件都会生成一个同名的.o 文件，该文件即为可重定位目标文件，最后所有.o 文件会链接为一个文件，即 Linux 内核。另外，静态链接库文件实际上就是可重定位文件的打包，也是可重定位文件，一般以.a作为文件名后缀。
- • 可执行文件：一般由多个可重定位文件结合生成，是完成了所有重定位工作和符号解析的文件（动态链接库符号是在运行时解析的），文件中保存着一个用来执行的程序。
- • 动态链接库文件：也称为共享目标文件，是已经经过链接处理可以直接加载运行的库文件，是可以被可执行文件或其他动态链接库文件加载使用的库文件，例如标准 C 的库文件 libc.so。可以简单理解为没有主函数 main 的“可执行”文件，只有一堆函数可供其他程序调用。Linux 下动态链接库文件文件名后缀为.so 的文件，so 代表 shared object。

ELF 概述

- ELF 文件的作用。ELF 文件参与程序的链接(构建一个可执行程序)和程序的执行(加载可执行程序)，所以可以从不同的角度来看待 ELF 格式的文件。
- • 如果用于编译和链接(可重定位文件)，则编译器和链接器将把 ELF 文件看作节的集合，所有节由节头表描述，程序头表可选。
- • 如果用于加载执行(可执行文件)，则加载器将把 ELF 文件看作程序头表描述的段的集合，一个段可能包含多个节和节头表可选。
- • 如果是动态链接库文件，则两者都含有

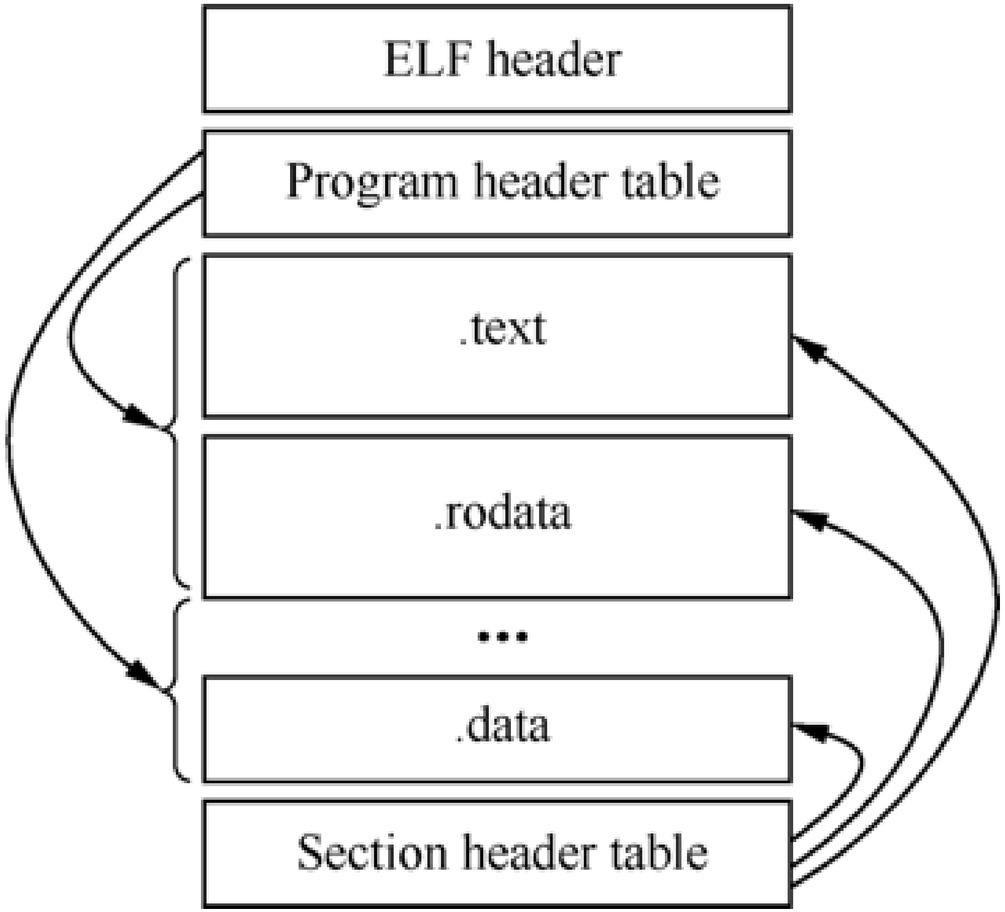
ELF 格式简介

- 我们首先简要介绍 ELF 文件格式，帮助您形成整体了解，接着选择性地详细讲解细节，以便您更好地理解可执行文件中存储的内容，以及这些内容是如何被加载到内存中的。
- 我们继续使用到如下的示例程序 hello.c
- 您可使用如下指令将其编译为一个32位或64位静态链接的ELF可执行文件hello。

```
#include<stdio.h>
void main()
{
    printf("Hello world!\n");
}

gcc -o hello hello.c -static
# 64 位机器上编出 32 位代码需加-m32
sudo apt-get install gcc-multilib
gcc -o hello hello.c -static -m32
```

ELF文件格式

- ELF文件的索引表。ELF文件的主体是各种节，典型的如代码节.text，还有描述这些节属性的信息（Program header table和Section header table），以及ELF文件的整体描述信息（ELF header），整体如图所示。



程序的编译过程

- 程序从源代码到可执行文件的编译步骤大致分为：预处理、编译、汇编、链接。以下示例继续使用hello.c，4步分别对应的指令如下。

```
# 预处理
gcc -E hello.c -o hello.i
# 编译
gcc -S hello.i -o hello.s
# 汇编
gcc -c hello.s -o hello.o
# 链接，-static为静态链接
gcc hello.o -o hello -static
```

汇编后形成的.o格式的文件已经是ELF格式文件

- 汇编后形成的.o格式的文件已经是ELF格式文件了。程序编译后生成的目标文件至少含有3个节区（Section），分别为.text、.data和.bss。在此为兼顾传统的名称，后面也称其为段。但要注意在ELF格式中Section与Segment是不同的。

- ❑ .bss段。BSS段（bss segment）通常是指用来存放程序中未初始化的全局变量的一块内存区域。BSS是BlockStarted by Symbol的简称。BSS段属于静态内存分配，该节区包含了在内存中的程序未初始化的数据。当程序开始运行时，系统将用0来初始化该区域。该节不占用文件空间，该section type = SHT_NOBITS。

- ❑ .data段。数据段（data segment）通常是指用来存放程序中已初始化的全局变量的一块内存区域。数据段属于静态内存分配。

- ❑ .text段。代码段（code segment/text segment）通常是指用来存放程序执行代码的一块内存区域。这部分区域的大小在程序运行前就已经确定，并且内存区域通常属于只读，某些架构也允许代码段为可写，即允许修改程序。在代码段中，也可能包含一些只读的常数变量，例如字符串常量等。

链接

- 链接是将各种代码和数据部分收集起来并组合成为一个单一文件的过程，这个文件可被加载（或被复制）到内存中并执行。在本例中就是将编译输出的.o文件与libc库文件进行链接，生成最终的可执行文件。
- 通俗地说，链接就是把多个文件拼接到一起，本质上是节的拼接。
- 比较链接前目标文件的节区信息表，可执行文件的节区信息表中节多了，由15个变成31个（本例中只显示了部分）。一个重要变化是，.text的Addr有了值。多出来的节是从外部库中添加过来的，编译器进行了整合，并安排了地址布局。另一个变化是，链接后多了段头表（Program header table）。可执行文件的加载执行，其实是操作系统按照段头的指示，将可执行文件按照安排好的布局加载到内存，再跳转到其中的代码段。
- readelf -S hello.o / readelf -S hello

链接与库

- 在可执行文件的生成过程中，最后的部分就是链接，链接对于我们理解可执行程序的加载和执行非常关键。链接从过程上讲分为符号解析和重定位两部分；根据链接时机的不同，又分为静态链接和动态链接两种。
- 先以hello.c为例简要说明符号、符号解析与重定位。其实应该是以hello.i为例，因为真正编译的C源文件是hello.i。
- 简单来说，hello.c中只有两个符号——main和printf。
- main的实现就在hello.c中，而printf的实现显然没有在hello.c中。相应的hello.c编译为hello.o后，main这个符号是“有定义”的，printf这个符号则是“无定义”的。
- “有定义”的意思就是函数对应的机器指令地址在当前文件中（有明确的地址）。
- 编译器需要到其他的共享库中找到printf的“定义（机器指令片段）”，找到后把该片段机器指令与hello.o拼接到一起（静态链接），生成可执行文件hello。hello中printf就存在了（有定义即有了明确的地址），这就是符号解析。
- 在拼接所有目标文件的同时，编译器会确定各个函数加载到内存中的地址，然后反过来修改所有调用该函数的机器指令，使得该指令能跳转到正确的内存地址。这个过程就是**重定位**。

符号与符号解析

- 符号包含全局变量和全局函数。例如printf就是一个符号，hello程序需要在函数库中找到这个符号。

- 符号表（symbol table）是一种供编译器用于保存有关源程序构造的各种信息的数据结构。这些信息在编译器的分析阶段被逐步收集并放入符号表，它们在综合阶段用于生成目标代码。符号表的每个条目包含与一个标识符相关的信息，比如它的字符串、类型、存储位置和其他相关信息。符号表通常需要支持同一标识符在一个程序中的多重声明。

- 符号表的功能是找未知函数在其他库文件中的代码段的具体位置。还是以hello.c为例，其调用的printf是外部库提供的函数。在链接前，编译器需要把类似于printf这种符号都记录下来，存储于符号表中。

- 符号表的查看方法为objdump -t hello.o或readelf -s hello.o。

符号与符号解析

- 符号包含全局变量和全局函数。例如printf就是一个符号，hello程序需要在函数库中找到这个符号。

- 符号表（symbol table）是一种供编译器用于保存有关源程序构造的各种信息的数据结构。这些信息在编译器的分析阶段被逐步收集并放入符号表，它们在综合阶段用于生成目标代码。符号表的每个条目包含与一个标识符相关的信息，比如它的字符串、类型、存储位置和其他相关信息。符号表通常需要支持同一标识符在一个程序中的多重声明。

- 符号表的功能是找未知函数在其他库文件中的代码段的具体位置。还是以hello.c为例，其调用的printf是外部库提供的函数。在链接前，编译器需要把类似于printf这种符号都记录下来，存储于符号表中。

- 符号表的查看方法为objdump -t hello.o或readelf -s hello.o。

重定位

- 重定位是把程序的逻辑地址空间变换成进程线性地址空间的过程，也就是链接时对目标程序中指令和数据的地址修改的过程。
- 重定位表中的每一条记录都对应一个需要重定位的符号。汇编器将为可重定位文件中每个包含需要重定位符号的段都建立一个重定位表。
- 符号表记录了目标文件中所有的全局函数及其地址；重定位表中记录了所有调用这些函数的代码位置。在链接时，这两大类数据都需要逐一修改为正确的值。

静态链接与动态链接

- 静态链接。**在编译链接时直接将需要的执行代码复制到最终可执行文件中**，优点是代码的装载速度快，执行速度也比较快，对外部环境依赖度低。编译时它会把需要的所有代码都链接进去，应用程序相对比较大。缺点是如果多个应用程序使用同一库函数，会被装载多次，浪费内存。

- 动态链接。**在编译时不直接复制可执行代码，而是通过记录一系列符号和参数，在程序运行或加载时将这些信息传递给操作系统。**

- 操作系统负责将需要的动态库加载到内存中，然后程序在运行到指定的代码时，去共享执行内存中已经加载的动态库去执行代码，最终达到运行时链接的目的。优点是多个程序可以共享同一段代码，而不需要在内存/磁盘上存储多个副本。缺点是在运行时加载，可能会影响程序的前期执行性能，而且对使用的库依赖性较高，在升级时特别容易出现版本不兼容的问题。

动态链接

- 如前文中的hello就是静态链接的可执行文件。如果在编译时不加“-static”选项，则编译器会默认使用动态链接。如下动态链接的可执行文件hello.dynamic只有7452字节，**而静态链接版本hello大小约是其100倍**

- 动态链接分为可执行程序**装载时动态链接**和**运行时动态链接**，接下来将介绍这两种动态链接。

```
$ gcc hello.o -o hello.dynamic
$ ls -l hello*
-rwxr-xr-x 1 root root    7452 8月   8 16:33 hello.dynamic
-rwxr-xr-x 1 root root  727908 8月   8 08:21 hello
```

装载时动态链接

- 以下实例源码shlibexample.h与shlibexample.c是一个简单动态库的源码，只提供一个函数SharedLibApi()。使用如下指令可能将其编译成libshlibexample.so文件。
- $ gcc -shared shlibexample.c -o libshlibexample.so
- 只要将以上头文件和生成库文件放置在正确的目录下，就可以像调用printf一样调用SharedLibApi()。
- shlibexample.h / shlibexample.c的源码如下

```c
/*  FILE NAME            :  shlibexample.h     */
#ifndef _SH_LIB_EXAMPLE_H_
#define _SH_LIB_EXAMPLE_H_

#define SUCCESS 0
#define FAILURE (-1)

#ifdef __cplusplus
extern "C" {
#endif
/*
 * Shared Lib API Example
 * input    : none
 * output   : none
 * return   : SUCCESS(0)/FAILURE(-1)
 *
 */
int SharedLibApi();
#ifdef __cplusplus
}
#endif
#endif /* _SH_LIB_EXAMPLE_H_
```

```c
/*  FILE NAME            :  shlibexample.c  */
#include <stdio.h>
#include "shlibexample.h"
/*
 * Shared Lib API Example
 * input    : none
 * output   : none
 * return   : SUCCESS(0)/FAILURE(-1)
 *
 */
int SharedLibApi()
{
    printf("This is a shared libary!\n");
    return SUCCESS;
}
```

运行时动态链接

- 运行时动态链接库的源文件为dllibexample.h和dllibexample.c。编译成libdllibexample.so文件的指令如下：
- gcc -shared dllibexample.c -o libdllibexample.so
- dllibexample.h / dllibexample.c的源码如下

```c
#ifndef _DL_LIB_EXAMPLE_H_
#define _DL_LIB_EXAMPLE_H_

#ifdef __cplusplus
extern "C" {
#endif
/*
 * Dynamical Loading Lib API Example
 * input    : none
 * output   : none
 * return   : SUCCESS(0)/FAILURE(-1)
 *
 */
int DynamicalLoadingLibApi();

#ifdef __cplusplus
}
#endif
#endif /* _DL_LIB_EXAMPLE_H_
```

```c
#include <stdio.h>
#include "dllibexample.h"


#define SUCCESS 0
#define FAILURE (-1)


/*
 * Dynamical Loading Lib API Example
 * input    : none
 * output   : none
 * return   : SUCCESS(0)/FAILURE(-1)
 *
 */
int DynamicalLoadingLibApi()
{
    printf("This is a Dynamical Loading libary!\n");
    return SUCCESS;
}
```

运行时动态链接

- //先将动态库加载进来
- void * handle = dlopen("libdllibexample.so",RTLD_NOW);
- //声明一个函数指针
- int (*func)(void);
- //根据名称找到函数指针
- func = dlsym(handle,"DynamicalLoadingLibApi");
- //调用已声明函数
- func();

动态链接实验

- 如下代码分别以装载时动态链接和运行时动态链接调用了两个动态链接库。从动态链接库的角度是没有差别的，差别只是程序员使用动态链接库的方法不同。

```
#include <dlfcn.h>

int main()
{
    printf("This is a Main program!\n");
    /* 装载时动态链接 */
    printf("Calling SharedLibApi() function of libshlibexample.so!\n");
    SharedLibApi();

    /* 运行时动态链接 */
    void * handle = dlopen("libdllibexample.so",RTLD_NOW);
    if(handle == NULL)
    {
        printf("Open Lib libdllibexample.so Error:%s\n",dlerror());
        return   FAILURE;
    }
    int (*func)(void);
    char * error;
    func = dlsym(handle,"DynamicalLoadingLibApi");
    if((error = dlerror()) != NULL)
    {
        printf("DynamicalLoadingLibApi not found:%s\n",error);
        return   FAILURE;
    }
    printf("Calling DynamicalLoadingLibApi() function of libdllibexample.so!\n");
    func();
    dlclose(handle);
    return SUCCESS;
}
```

可执行程序装载概要

- 要研究可执行程序的装载，除了可执行文件的格式之外，还需要把执行环境的来龙去脉搞清楚。
- 了解可执行程序的执行环境，一般是Shell程序来启动一个可执行程序的，Shell程序做了什么。当 Shell 启动加载一个可执行程序时（发起fork和execve系统调用），准备了哪些执行的上下文环境。
- 这样大概了解了用户态的执行环境了，再来看一个系统调用怎么把一个可执行文件在内核里装载起来又返回到用户态。

用户态程序执行环境上下文

- 还是以一个例子开始，如果在Shell中输入如下指令ls -l /usr/bin，实际上执行了可执行程序ls，后面带两个参数-l和/usr/bin。与前文./hello执行不带参数自己写的可执行文件hello并无二致。Shell本身不限制命令行参数的个数，命令行参数的个数受限于命令自身，也就是main函数愿意接收什么。典型的main函数可以写成如下几种：

- int main()

- int main(int argc，char*argv[])

- int main (int argc，char *argv[]，char *envp[])

- 前两种比较常见，第三种是如果愿意接收Shell的执行环境，还可以再加一个char *envp[]，一般Shell程序会自动为可执行文件加上执行环境。用户在输入命令时，比如ls –l /usr/bin中的–l与/usr/bin是两个参数，会通过int argc和char *argcv[]传进来。这是Shell命令传递到可执行程序的方法。

execve系统调用接口函数

- Shell会调用execve系统调用接口函数将命令行参数和环境变量传递给可执行程序的main函数。execve系统调用接口函数的函数原型如下：
- int execve(const char *filename, char *const argv[],char *const envp[]);
- filename为可执行文件的名字，argv是以NULL结尾的命令行参数数组，envp同样是以NULL结尾的环境变量数组（使用命令man execve，可查看其说明）。编程使用的exec系列库函数都是execve系统调用接口函数的封装接口

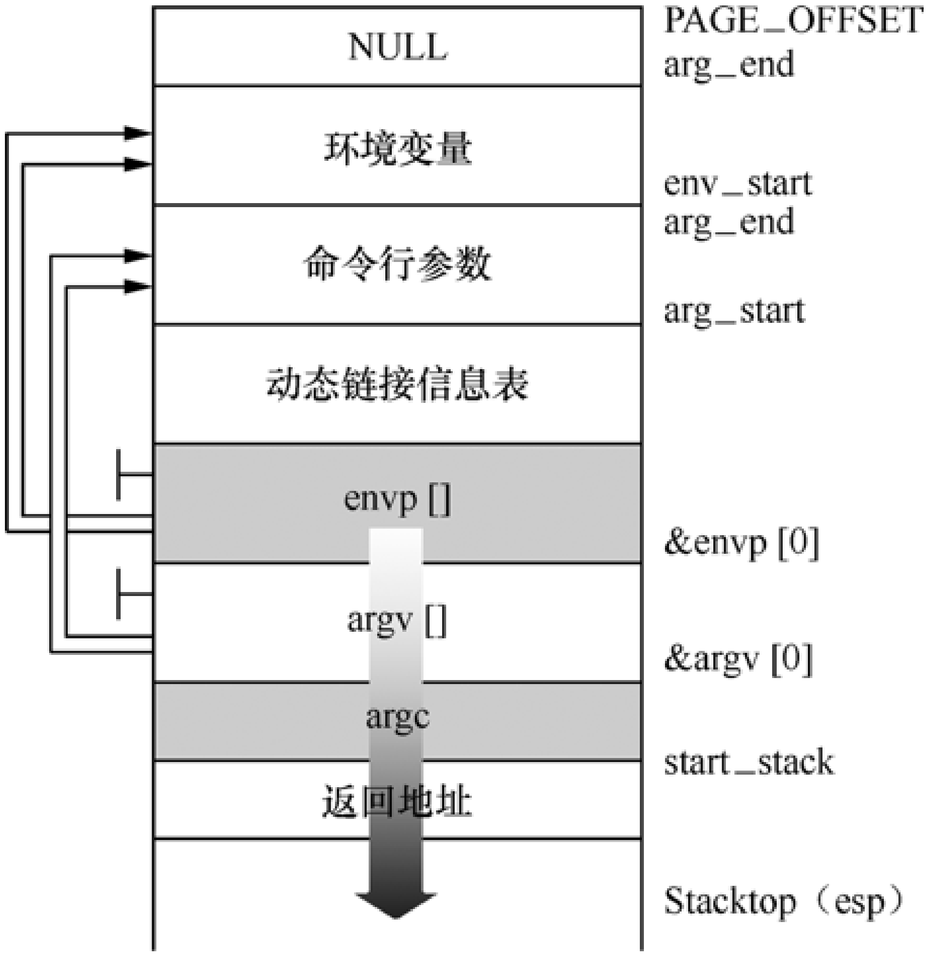
简化了的Shell程序

- 如上范例代码即为简化了的Shell程序执行ls命令的过程。首先fork一个子进程，pid为0的分支是将来的子进程要执行的，在子进程里调用execlp来加载可执行程序ls，这里没有写环境变量。完整的Shell程序中会有环境变量，接收与否则取决于子进程的main函数。Shell程序大致就是这样工作的。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(int argc, char * argv[])
{
    int pid;
    /* fork another process */
    pid = fork();
    if (pid < 0)
    {
        /* error occurred */
        fprintf(stderr, "Fork Failed!");
        exit(-1);
    }
    else if (pid == 0)
    {
        /*   child process   */
        execlp("/bin/ls", "ls", NULL);
    }
    else
    {
        /*     parent process  */
        /* parent will wait for the child to complete*/
        wait(NULL);
        printf("Child Complete!");
        exit(0);
    }
}
```

命令行参数和环境变量是如何保存的呢?

- 当fork一个子进程时，会生成子进程的进程描述符、内核堆栈和用户态堆栈等，子进程是通过复制父进程的大部分内容创建的。子进程通过execlp加载可执行程序时按如图所示的结构重新布局用户态堆栈，可以看到用户态堆栈的栈顶就是main函数调用堆栈框架，这就是程序的main函数起点的执行环境。



execve系统调用加载静态链接程序

- 在布局一个新的用户态堆栈时，实际上是把命令行参数内容和环境变量的内容通过指针的方式传到系统调用内核处理函数，再创建一个新的用户态堆栈时会把这些char *argcv[]和char *envp[]等复制到用户态堆栈中，来初始化这个新的可执行程序的执行上下文环境。所以新的程序可以从main函数开始把对应的参数接收过来，然后执行。

- 值得注意的是，在调用execve系统调用时，当前的执行环境是从父进程复制过来的，execve系统调用加载完新的可执行程序之后已经覆盖了原来父进程的上下文环境。execve系统调用在内核中帮我们重新布局了新的用户态执行环境。

- 执行readelf -h可以查看ELF可执行文件首部信息，如下所示程序入口点Entry point address:0x804887f。如果是静态链接程序在execve系统调用加载完成后，堆栈上的返回地址会修改为程序入口点的地址。当系统调用从内核态返回时，会从该地址0x804887f继续执行。

execve系统调用加载动态链接程序

- 如果仅加载一个静态链接可执行程序，只需要传递一些命令行参数和环境变量，就可以正常工作了。但对于绝大多数可执行程序来讲，还有一些对动态链接库的依赖会稍微复杂一点。
- 动态链接的程序从内核态返回时首先会执行.interp节区指向的动态链接器。通过readelf -l/S可以看到，动态链接的可执行文件会比静态链接多出.interp这个节以及其他ld需要用到的节，段表也会有相应的INTERP段。
- readelf -p.interp查看节的内容，可以看到节中存储的是动态链接器的完整路径。
- 一个动态链接库还可能会依赖其他的动态链接库，这样形成了一个依赖关系图——动态链接库会生成依赖树，ldd命令查看/bin/ls程序为例，可以看到ls所依赖动态链接库。
- 动态链接器ld负责加载动态链接库，这是一个依赖关系图的遍历，从而装载所有需要的动态链接库，动态链接完成后ld将CPU的控制权交给可执行程序入口。动态链接的过程主要是动态链接器负责，而不是Linux内核完成的。

execve系统调用的内核处理过程

- 有了前面的知识储备，我们可以想象一下可执行文件的加载过程。因为所有的信息都包含在ELF文件中，什么段放在什么内存区域，按照ELF的要求放好即可。当然这其中涉及很多细节：修改进程描述符、申请内存、堆栈、准备参数、跳转到入口点等。接下来我们梳理一下execve系统调用的内核处理过程。

- Linux系统一般会提供了execl、execlp、execle、execv、execvp和execve等6个用以加载执行一个可执行文件的库函数，这些库函数统称为exec函数，差异在于对命令行参数和环境变量参数的传递方式不同。exec函数都是通过execve系统调用进入内核，对应的系统调用内核处理函数为sys_execve或__x64_sys_execve，它们都是通过调用do_execve来具体执行加载可执行文件的工作。

- 整体的调用关系为sys_execve()或__x64_sys_execve -> do_execve() –> do_execveat_common() -> __do_execve_file -> exec_binprm()-> search_binary_handler() -> load_elf_binary() -> start_thread()。

search_binary_handler

```
static LIST_HEAD(formats);
...
/*
 * cycle the list of binary formats handler, until one recognizes the image
 */
int search_binary_handler(struct linux_binprm *bprm)
{
...
    list_for_each_entry(fmt, &formats, lh) {
        if (!try_module_get(fmt->module))
            continue;
        read_unlock(&binfmt_lock);
        bprm->recursion_depth++;
        retval = fmt->load_binary(bprm);
```

search_binary_handler

- 核心代码与众多文件格式解耦合的方法，和面向对象设计中的多态本质上是一致的。

```
static struct linux_binfmt elf_format = {
    .module        = THIS_MODULE,
    .load_binary    = load_elf_binary,
    .load_shlib    = load_elf_library,
    .core_dump    = elf_core_dump,
    .min_coredump    = ELF_EXEC_PAGESIZE,
};
...
static int __init init_elf_binfmt(void)
{
    register_binfmt(&elf_format);
    return 0;
}
...
core_initcall(init_elf_binfmt);
```

load_elf_binary

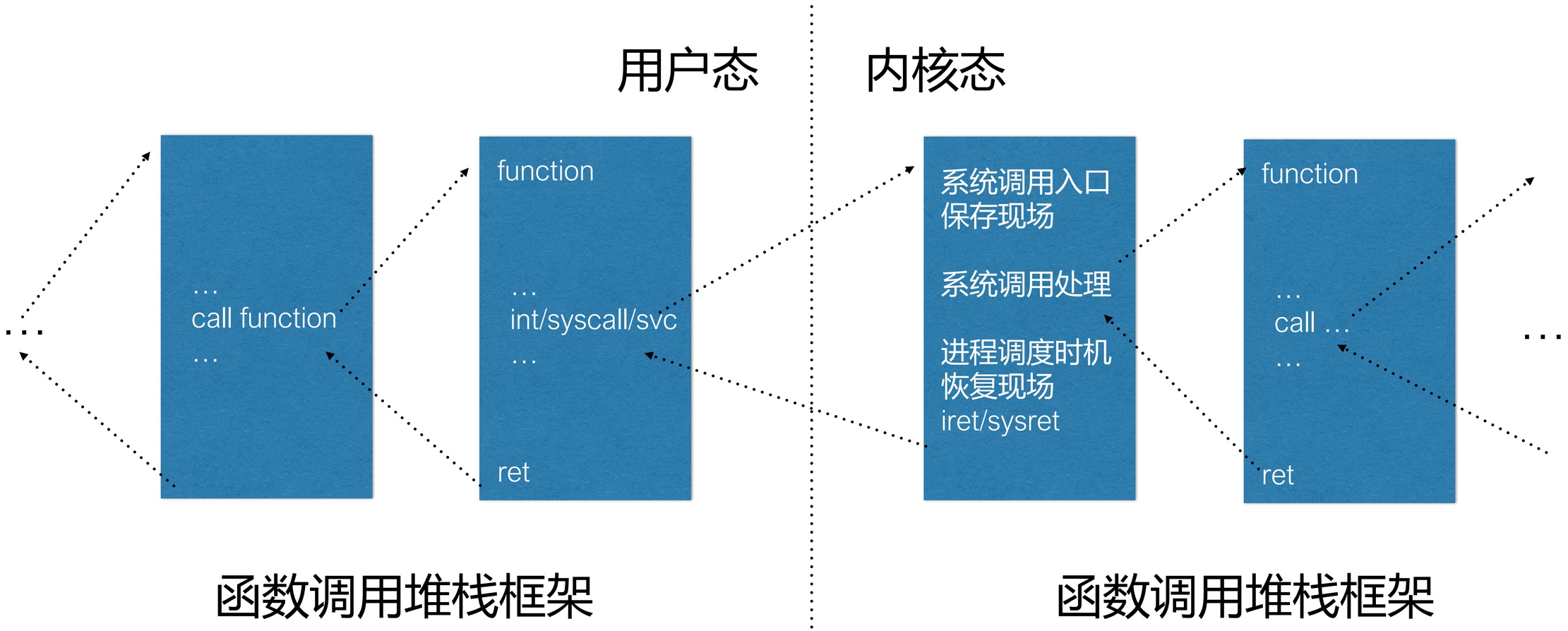
- 校验文件
- 加载文件到内存并根据ELF文件中（Program header table和Section header table）映射到进程的地址空间
- 判断是否需要动态链接
- 配置进程启动上下文环境start_thread

ARM64 start_thread

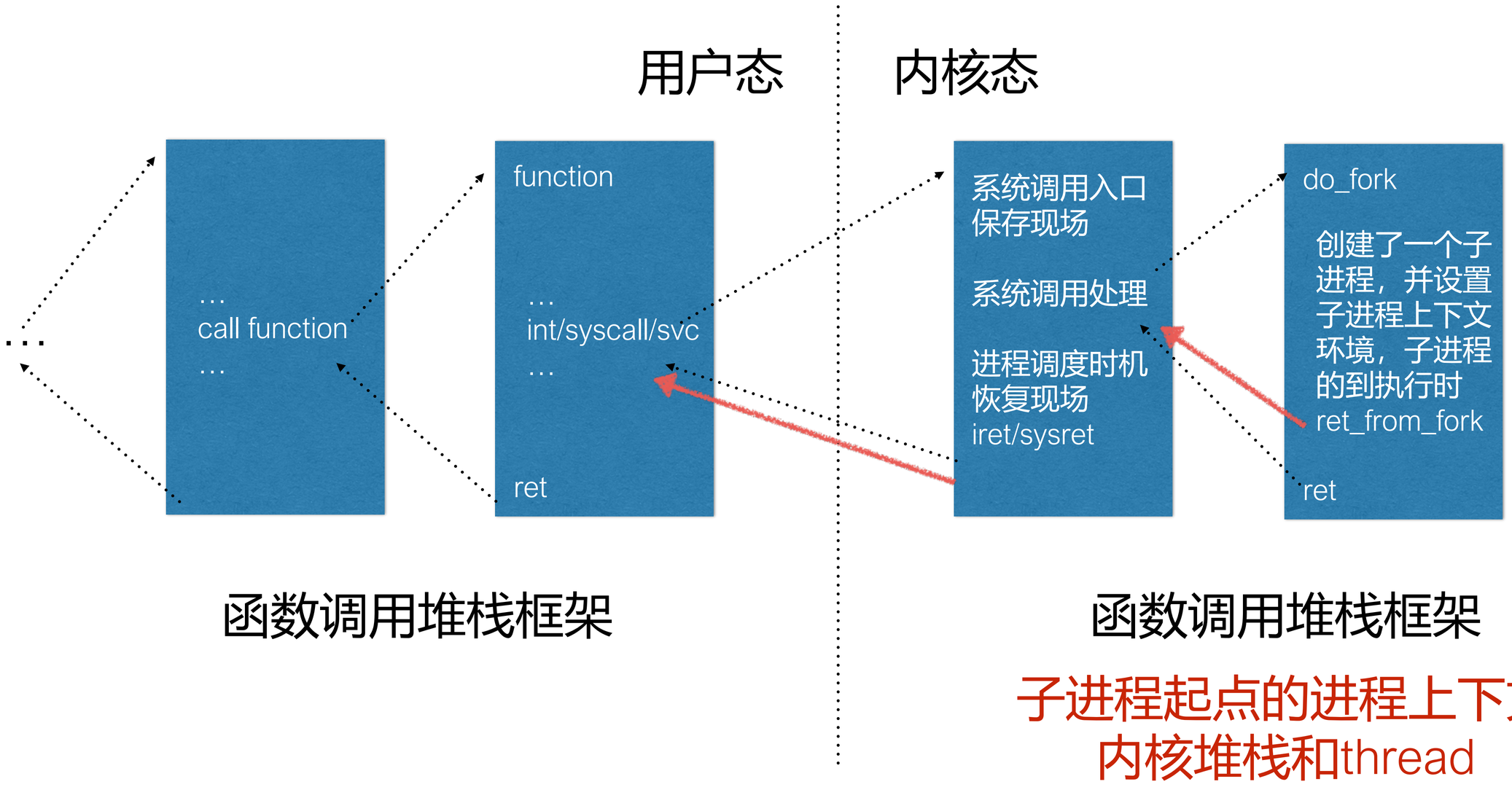
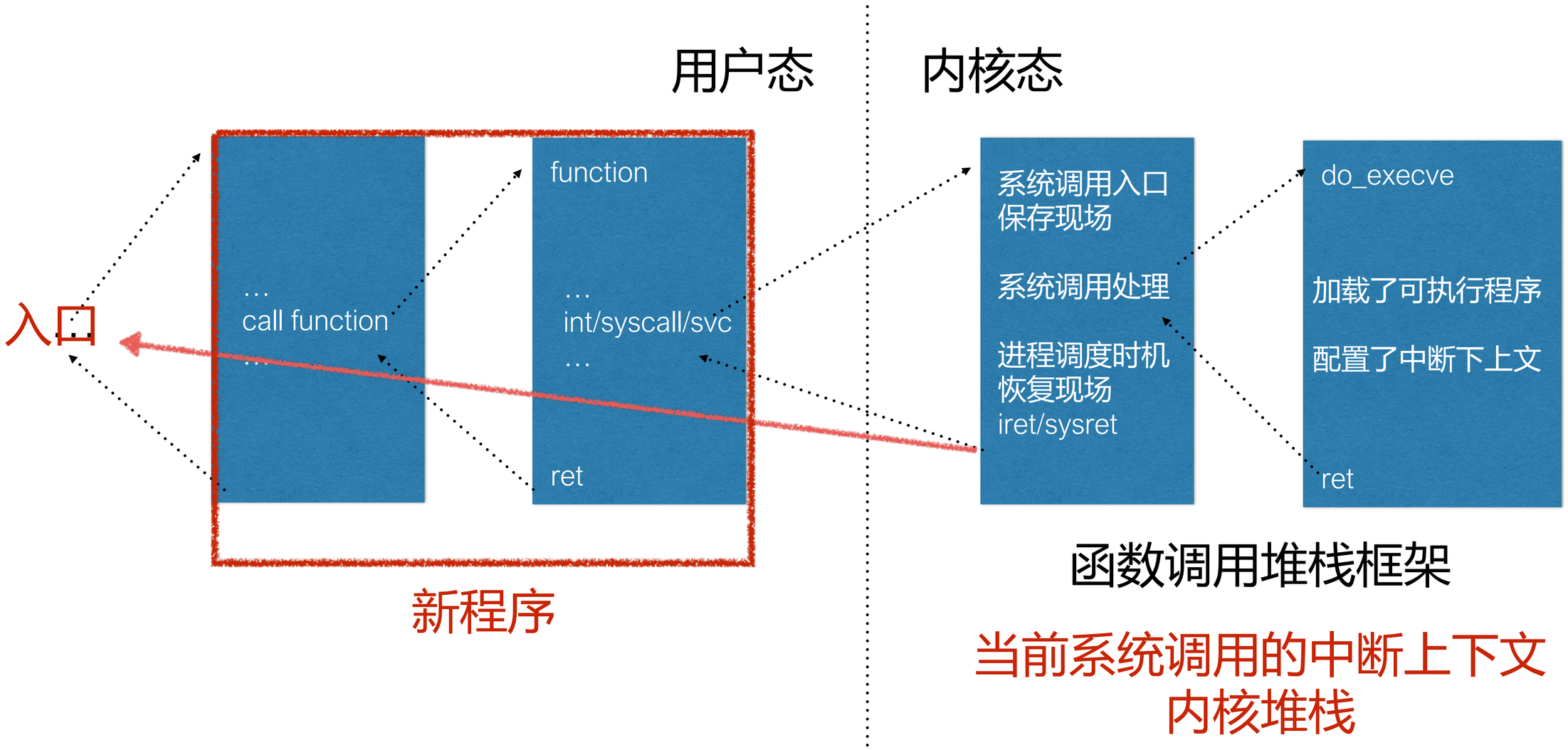
```
static inline void start_thread_common(struct pt_regs *regs, unsigned long pc)
{
    memset(regs, 0, sizeof(*regs));
    forget_syscall(regs);
    regs->pc = pc;

    if (system_uses_irq_prio_masking())
        regs->pmr_save = GIC_PRIO_IRQON;
}
...
static inline void start_thread(struct pt_regs *regs, unsigned long pc,
                unsigned long sp)
{
    start_thread_common(regs, pc);
    regs->pstate = PSR_MODE_EL0t;

    if (arm64_get_ssbd_state() != ARM64_SSBD_FORCE_ENABLE)
        set_ssbs_bit(regs);

    regs->sp = sp;
}
```

可以看到start_thread修改了内核堆栈的底部，即中断关键上下文的CPU状态信息，使得execve系统调用返回到用户态时能够从新的程序入口开始执行。

系统调用、fork和execve



系统调用、fork和execve



系统调用、fork和execve